

C++ Runtime Polymorphism, Upcasting, and Virtual Mechanism

1. Upcasting

Definition:

Upcasting refers to assigning the address of a **derived class object** to a **base class pointer** or **reference**.

Rules and Conditions

1. **Inheritance mode must be public.**
 - In **private** or **protected** inheritance, upcasting is not allowed outside the class hierarchy.
2. Upcasting can be done using either:
 - **Pointer**, or
 - **Reference** to base class.
3. Once upcasting occurs, only **base class members** (accessible ones) can be used.
4. For **non-virtual functions**, the **function of the class whose pointer/reference type** is used gets called.
5. For **virtual functions**, the **function of the class whose object** is pointed to gets called.

Example

2. Binding (Function Resolution)

Definition:

Binding means associating a function call with its actual function body.

Types of Binding

1. **Early Binding (Static Binding)**
 - Default in C++.
 - Happens for **non-virtual** functions.
 - The compiler binds the call at compile time based on **type of pointer or reference**, not the object.
 - Used when the function call can be resolved at compile time.
2. **Late Binding (Dynamic Binding)**
 - Happens only when:
 - The function is declared **virtual** in the base class, **and**
 - It is called through a **base class pointer or reference**.
 - The compiler defers the call until runtime and executes based on **content of pointer (actual object)**.

Summary

Function Type	Pointer Type	Binding	Function Executed
---------------	--------------	---------	-------------------

Non-virtual	Base*	Early	Base function
Virtual	Base*	Late	Derived function

3. Redefinition vs Overriding

Concept	Virtual Keyword	Binding	Rule	Behavior
Redefinition	Absent	Early	Same name in child hides base version	Determined by pointer type
Overriding	Present	Late	Base function is virtual and redefined in child	Determined by object type

Compiler's Action:

When you override a parent class method in the child:

- Even if you make it **private**, overriding is valid.
- The compiler checks:
 1. The **content of the pointer/reference** (actual object type),
 2. Whether the base class version of the function is **virtual and accessible**.
- If so, compiler sets up runtime dispatch using **VTABLE** and **VPTR**.

4. Dynamic Polymorphism – 3 Ways

Runtime polymorphism (late binding) can be achieved in **three ways**:

1. Using Base Class Pointer

```
Base* ptr = new Derived;
ptr->disp();
```

2. Using Base Class Reference

```
Derived d;
Base& ref = d;
ref.disp();
```

3. Using Array of Base Class Pointers

```
Animal* arr[3];
arr[0] = new Dog;
arr[1] = new Cat;
arr[2] = new Tiger;
```

```
for (int i = 0; i < 3; i++) arr[i]->makeSound();
```

Here, each pointer stores address of a derived object, and function calls resolve dynamically.

5. Virtual Function

Definition:

A virtual function is a base class function declared with the keyword **virtual**, allowing **derived classes** to override it and enabling **runtime polymorphism**.

Key Points

1. Virtual functions work **only via base class pointer or reference**.

2. The compiler ensures **late binding** only when the function is virtual.
3. If a class has at least one virtual function, it is automatically **inherited as virtual**.
4. Virtuality is maintained across derived classes unless the function is re-declared as final or non-virtual.
5. Even if the derived class **does not override**, it still receives a **VTABLE** containing the parent's virtual function address.
6. The **address order** of virtual functions in the table depends on their **declaration order** in the class.

6. Virtual Table (VTABLE) and VPTR Mechanism

When a class contains at least one virtual function, the compiler automatically generates a **Virtual Table (VTABLE)** for that class.

Conceptual Layout

Each object of a class with virtual functions has a **hidden data member** called **VPTR (Virtual Pointer)** that points to its class's VTABLE.

```
class Base {
public:
    virtual void disp();
};

class Child : public Base {
public:
    void disp() override;
};
```

Compiler Actions

1. Compiler creates **one VTABLE per class**.
2. Each entry in VTABLE stores the **address of a virtual function**.
3. Each object has a hidden **VPTR**, initialized by constructor.
4. When a virtual function is called:
 - Compiler checks the **content of pointer** (object type).
 - Fetches the **VPTR** from the object.
 - Accesses the **VTABLE** through VPTR.
 - Invokes the corresponding function using the **stored address**.

Visualization

Class	Object Addr	VPTR	VTABLE Entries (Function Addresses)
Base	1000	→ 5000	disp() → 3000
Child	1300	→ 9000	disp() → 4000

If child doesn't override, the compiler fills its VTABLE entry with the **base function address**.

Multiple Virtual Functions Example

```
class Base {
public:
```

```

        virtual void disp1();
        virtual void disp2();
    };
    class Child : public Base {
    public:
        void disp2() override;
        virtual void disp3();
    };

```

VTABLE Layout:

Function Index	Base::VTABLE	Child::VTABLE
0	disp1 → Base::disp1	disp1 → Base::disp1
1	disp2 → Base::disp2	disp2 → Child::disp2
2	—	disp3 → Child::disp3

Order of entries in VTABLE strictly follows the **declaration order** in the base class.

7. Virtual Destructors

When deleting a derived object through a base pointer:

```

Base* b = new Derived;
delete b;

```

If the base destructor is **not virtual**, only the **base class destructor** executes, leading to **memory leaks** if the derived class allocates dynamic memory.

Solution:

Declare **virtual destructor** in the base class.

```

class Base {
public:
    virtual ~Base() { cout << "Base destroyed\n"; }
};
class Derived : public Base {
public:
    ~Derived() { cout << "Derived destroyed\n"; }
};

```

Output:

Derived destroyed

Base destroyed

Compiler Behavior:

At delete b;, compiler checks the **content of pointer**, finds VPTR of Derived, fetches destructor address from VTABLE, and executes destructors in correct order.

8. Virtual Function Calls from Constructors and Destructors

When a virtual function is called **inside a constructor or destructor**, **early binding** occurs.

Reason:

During construction or destruction, the object's **VPTR is still bound to the current class**, not to the most-derived class.

```

class Base {
public:
    Base() { disp(); } // Calls Base::disp()

```

```

        virtual void disp() { cout << "Base disp\n"; }
    };
    class Derived : public Base {
    public:
        void disp() override { cout << "Derived disp\n"; }
    };

```

Output:

Base disp (because Derived part not yet constructed).

Same rule applies in destructors.

9. Virtual Functions Called from Member Functions

If a **member function** calls a virtual function:

- **Called by object:** → Early binding
- **Called by pointer/reference:** → Late binding

```

class Base {
public:
    virtual void disp() { cout << "Base disp\n"; }
    void run() { disp(); } // Depends on caller
};
class Child : public Base {
public:
    void disp() override { cout << "Child disp\n"; }
};
int main() {
    Base b; b.run(); // Early → Base disp
    Child c; c.run(); // Early → Child disp
    Base* p = new Child;
    p->run(); // Late → Child disp
}

```

10. Internal Structure of VTABLE

The VTABLE stores:

1. **Addresses of all virtual functions.**
2. **Address of virtual destructor** (if present).
3. **Information about pure virtual functions.**
4. **RTTI (Run-Time Type Information)** entries for type identification.

Each class gets **one VTABLE**, each object gets **one VPTR per base class containing virtuals**.

11. Notes on Multiple Inheritance

If multiple base classes contain virtual functions:

- Each base class contributes its own **VTABLE** and **VPTR**.
 - The derived object stores **multiple VPTRs**, one for each base.
-

12. Virtual Constructor (Factory Concept)

Constructors cannot be declared virtual because object creation type must be known at compile time. However, the behavior can be mimicked using a **Factory Method**.

```
class Shape {
public:
    virtual void draw() = 0;
    static Shape* createShape(int choice);
};
Shape* Shape::createShape(int choice) {
    switch (choice) {
        case 1: return new Circle;
        case 2: return new Triangle;
        default: return nullptr;
    }
}
```

This pattern is informally known as a **virtual constructor**.

13. Summary Table

Concept	Meaning	Binding Type	Notes
Upcasting	Derived → Base	Depends	Enables polymorphism
Downcasting	Base → Derived	Explicit (dynamic_cast)	May fail if object not derived
Redefinition	Non-virtual same name	Early	Hides base version
Overriding	Virtual same signature	Late	Enables dynamic binding
Virtual Function	Late-bound member	Runtime	Declared with virtual
Virtual Destructor	Destroys derived correctly	Runtime	Always use if base pointer delete
VTABLE	Stores function addresses	Runtime	One per class
VPTR	Pointer to VTABLE	Runtime	One per object
Virtual Constructor	Factory method	Runtime	Creates derived via base pointer
RTTI	Runtime type identification	Runtime	Stored in VTABLE
Object Slicing	Derived → Base object copy	Compile time	Loses derived data

Vtable in Multiple Inheritance

1. Multiple Inheritance Case

Now suppose the derived class inherits **two base classes**, both having virtual functions:

```
class A(has virtuals)
class B(has virtuals)
class C : public A, public B
```

Then the **object layout** changes as follows:

[C Object Layout]

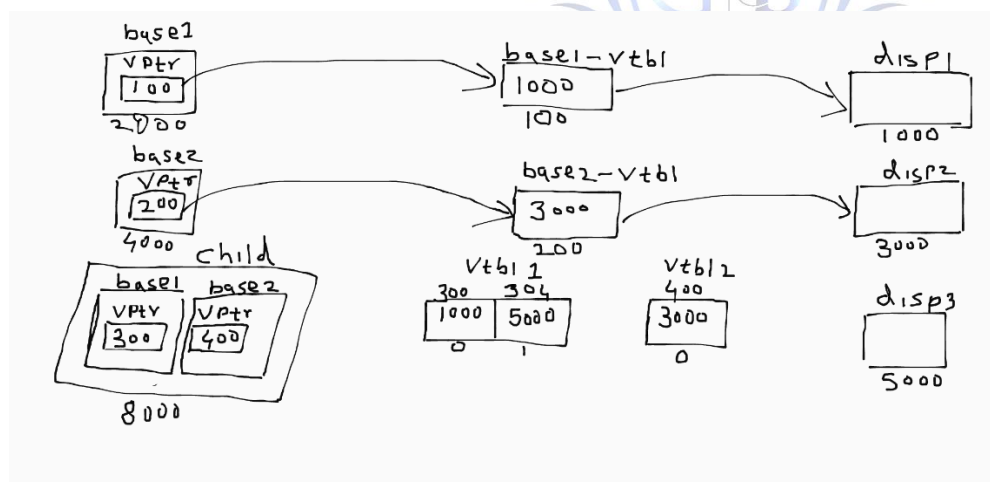
vptr_A-- > vtable of C(for A part)	
Data members of A	
vptr_B-- > vtable of C(for B part)	
Data members of B	
Data members of C	

Key Idea:

- Each **parent subobject** in C gets **its own vptr** because both A and B have their own virtual function mechanisms.
- So, C's object has **two vptrs** — one for each base part.
- Each vptr points to its respective **vtable** segment, which knows how to call the correct virtual function versions.

2. Behavior of VTABLE Entries

Let's visualize the **vtable structure** conceptually:



Explanation:

- The **first vtable** contains entries of all virtual functions coming from A.
- The **second vtable** contains entries of all virtual functions coming from B.
- If C overrides a virtual function from either base, the entry in that respective vtable now points to C's overridden method.
- If not overridden, the entry remains pointing to the parent's version.

3. Order of vptrs in the Object

The **order in which vptrs appear** inside the object is **the same as the order of base class inheritance**:

Example:

```
class C : public A, public B →  
vptr_A comes first, then vptr_B.
```

If inheritance order were class C : public B, public A,
then vptr_B would appear first, then vptr_A.

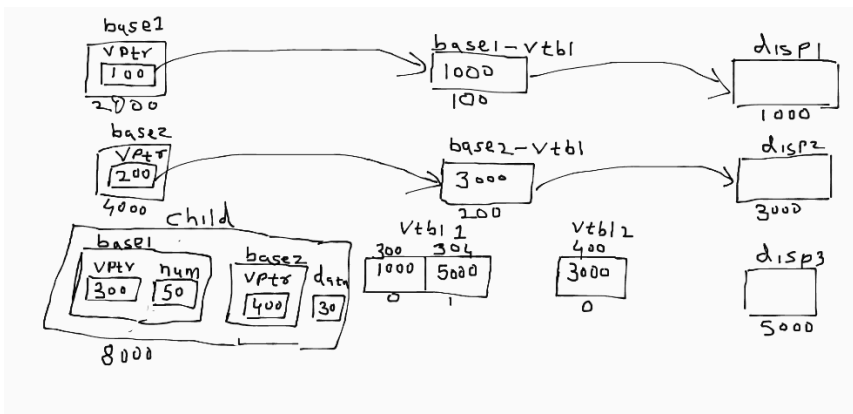
So:

```
[C:public A, public B]  
Object Layout :  
| vptr_A | data of A | vptr_B | data of B | data of C |
```

```
[C:public B, public A]  
Object Layout :  
| vptr_B | data of B | vptr_A | data of A | data of C |
```

4. Combined View

Let's visualize the **complete conceptual structure**:



Abstract Class in C++

1. Definition

- An **abstract class** in C++ is a class that **contains at least one pure virtual function**.
 - It serves as a **blueprint** or **contract** for derived classes.
-

2. Pure Virtual Function

- A **pure virtual function** is a function that is **declared** but **not defined** in the base class.
- It represents a **contract** that all derived classes **must implement**.

Syntax:

```
virtual <return_type> function_name(<parameters>) = 0;
```

3. Characteristics of Abstract Class

1. Instantiation Restriction

- An abstract class **cannot be instantiated** directly.
- This is because it contains at least one function with **no implementation**, making the class **incomplete**.

2. Pointer and Reference Allowed

- You **can create pointers and references** to an abstract class.
- These can point or refer to objects of derived classes.

3. Constructor in Abstract Class

- An abstract class **can have a constructor**.
- Although the class cannot be instantiated directly, the **constructor will be invoked** when an object of its **child class** is created.

4. Function Types Allowed

- An abstract class can contain:
 - **Pure virtual functions** (contracts)
 - **Virtual functions** (which can be overridden)
 - **Non-virtual functions** (regular member functions)
-

4. Purpose of Abstract Class

- To **define a general interface** (contract) for derived classes.
- To **enforce** that every derived class **implements specific functions** (pure virtual functions).

- To **provide shared implementation** for common behaviors (through non-virtual or virtual functions).

5. Example Explanation

```
#include <iostream>
using namespace std;

class Person // Abstract class
{
public:
    // Constructor (allowed)
    Person() {
        cout << "Person constructor invoked\n";
    }

    // Pure virtual function (contract)
    virtual void performDuties() = 0;

    // Virtual function (can be overridden if needed)
    virtual void talk() {
        cout << "Person talks politely\n";
    }

    // Non-virtual functions (concrete behaviour)
    void walk() {
        cout << "Person walks\n";
    }

    void eat() {
        cout << "Person eats\n";
    }

    void sleep() {
        cout << "Person sleeps\n";
    }
};

// Derived class 1
class Teacher : public Person
{
public:
    Teacher() {
        cout << "Teacher constructor invoked\n";
    }

    // Overriding the pure virtual function
    void performDuties() override {
        cout << "Teacher teaches students\n";
    }

    // Overriding the virtual function (optional)
    void talk() override {
        cout << "Teacher explains concept clearly\n";
    }
};

// Derived class 2
class Soldier : public Person
{
public:
    Soldier() {
        cout << "Soldier constructor invoked\n";
    }

    void performDuties() override {
        cout << "Soldier protects the nation\n";
    }
};

// Derived class 3
class HouseWife : public Person
{

```

```

public:
    HouseWife() {
        cout << "HouseWife constructor invoked\n";
    }

    void performDuties() override {
        cout << "HouseWife manages home\n";
    }
};

// Function using base class reference
void perform(Person& p) {
    p.performDuties();
    p.talk();
    p.walk();
    p.eat();
    p.sleep();
}

int main() {
    // Abstract class cannot be instantiated
    // Person p;    // X Error

    // But base class pointer or reference can be used
    Person* p1 = new Teacher;
    Person* p2 = new Soldier;
    Person* p3 = new HouseWife;

    cout << "\n--- Using Base Class Pointers ---\n";
    p1->performDuties();
    p2->performDuties();
    p3->performDuties();

    cout << "\n--- Using Reference Function ---\n";
    Teacher t;
    Soldier s;
    perform(t);
    perform(s);

    // Cleanup
    delete p1;
    delete p2;
    delete p3;

    return 0;
}

```



Explanation

Concept	Demonstrated In
Pure virtual function	performDuties() in Person
Virtual function	talk() (can be optionally overridden)
Non-virtual functions	walk(), eat(), sleep()
Abstract class	Person
Instantiation restriction	Person cannot be instantiated directly
Constructor in abstract class	Person() — executed when derived object is created
Polymorphism via pointer	Person *p1 = new Teacher; etc.
Polymorphism via reference	perform(Person &p)
Runtime dispatch	Calls correct overridden performDuties() based on object type

Program to Interface and Loose Coupling

1. Concept Overview

- **Programming to Interface** means designing classes and functions to depend on **abstract types (interfaces or base classes)** rather than concrete implementations.
 - This approach enables **loose coupling, flexibility, and ease of maintenance**.
-

2. Loose Coupling

- **Loose coupling** occurs when a class depends only on **abstract behavior** (e.g., base class or interface), not on a specific implementation.
- It allows replacing or extending implementations **without modifying the dependent class**.
- Example conceptually:
 - Soldier depends on Weapon (an abstract class).
 - Any derived weapon (Gun, Sword, Rocket) can be used interchangeably.
 - Only the reference to Weapon is stored inside Soldier, not any concrete object.

Advantages

1. **Maintenance advantage** – future changes in implementation require no modification in dependent class.
 2. **Extensibility** – new classes can be introduced without altering existing code.
 3. **Reusability** – components can be reused in other contexts with different implementations.
 4. **Testability** – easier to test by injecting mock objects.
-

3. Tight Coupling

- **Tight coupling** means a class depends directly on a **specific implementation**.
- Changing or replacing that dependency **requires modifying the dependent class** itself.
- Example conceptually:
 - Soldier directly contains a Gun object.
 - Replacing Gun with Sword requires editing the Soldier class.

Drawbacks

1. **Low flexibility** – cannot easily replace or extend functionality.
 2. **High maintenance cost** – any change in implementation affects multiple classes.
 3. **Poor reusability** – classes become bound to specific implementations.
-

4. Example Summary (Conceptual)

Design Approach	Dependency	Coupling Type	Impact of Change
Soldier → Weapon*	Depends on abstract class (Weapon)	Loose Coupling	No change needed if new weapon added
Soldier → Gun	Depends on specific implementation (Gun)	Tight Coupling	Must modify Soldier class

5. Program to Interface (vs) Program to Implementation

Concept	Description
Program to Implementation	Function or class directly creates or uses specific objects (e.g., Gun, Triangle). Leads to tight coupling .
Program to Interface	Function or class uses base class pointers/references (e.g., Weapon*, Shape*). Enables loose coupling .

1. Object Slicing

Definition:

Occurs when a **derived class object** is **assigned to a base class object** (not pointer/reference).

```
base b;  
sub s;  
b = s; // object slicing
```

Detailed Behavior

- Both b and s are separate objects stored in their own memory.
- Assignment b = s invokes the base class assignment operator (since b is of type base).
- The compiler allows this because a derived object *is-a* base object.
- However, the derived part of s (its extra data members and overridden methods) is not copied — only the base portion (i, j) is.
- After assignment, b contains only the base data; the extra members (k) and derived behavior are lost.
- Hence, s remains fully intact, but b holds only the sliced-off base portion.
- No runtime polymorphism can occur here, since b is a concrete base object — not a pointer or reference.

Reason:

- base object can only hold **base part** (its own members).
- The **base class constructor, copy constructor, and assignment operator** only know about base members.
- They cannot access or copy members of derived class because the base class definition is fixed.
- Thus, only base fields are assigned — this “cuts off” or *slices* the derived portion.

Result:

- Only the base members of s are copied into b.
 - b loses access to derived data and overridden behaviors.
-

2. Why It Happens

- The **base class's copy constructor or assignment operator** knows nothing about derived members.
- Hence only base portion gets copied.

3. Consequences

- Loss of **derived data** and **derived behavior**.
 - Causes incorrect results if virtual behavior or derived members are expected.
 - Leads to **maintenance and debugging issues** if polymorphism is mistakenly expected.
-

4. Avoiding Object Slicing

- Use **pointers or references** (upcasting) instead of assigning by value.
 - Use **virtual functions** to achieve correct runtime behavior.
-

In short:

- **Upcasting** → No data loss, enables polymorphism (via pointers/references).
 - **Object Slicing** → Happens when assigning objects by value; derived part is lost since only base portion is copied.
 - The data of base class which is modified in sub is assigned to the base class hence the only base class data is copied.
-

Operator Inheritance Concept

1. Overview

When a class is derived from a base class, **most overloaded operators are inherited**, but **operator= is not**.

This difference exists because operator= performs **constructor-like activity** (copying object data), and every class must manage its own data separately.

2. operator+ — Inherited

- operator+ behaves as a **regular member function**, so it is **inherited** by the derived class.
- The derived object can directly use the base version unless it redefines its own.

Example Behavior

```
base b1, b2;  
sub s1, s2;  
b1 + b2;    // base::operator+()  
s1 + s2;    // inherited base::operator+()
```

Explanation:

- The operator+ inherited from the base operates only on **base members** because it knows only the base class layout.
 - Even if derived objects have extra members, they are **ignored** during addition, as the operator only accesses members of the base part.
-

3. operator= — Not Inherited

- Treated specially by the compiler.
 - **Not inherited** because each class must control how its members are copied.
 - If not explicitly defined in the derived class, the compiler **auto-generates** one that:
 - Copies derived data.
 - Calls `base::operator=` internally to copy the base part.
-

4. Why operator= Is Not Inherited

1. The base operator knows only base members.
 2. The derived class may have extra members that must also be copied.
 3. Hence, the compiler generates a unique operator= for every class that:
 - Copies derived members, and
 - Calls the base operator= automatically.
-

5. Behavior Summary

Case	Description	Behavior
No operator= in sub	Only in base	Compiler creates one that calls base's operator=
operator= defined in sub (no base call)	Overrides base version	Only derived data copied; base data remains unchanged
operator= defined in sub and calls base::operator=()	Full copy	Both base and derived members copied
operator+ in base	Normal operator	Inherited and callable by derived

6. Conceptual Flow

A. Compiler-generated

```
sub::operator=(sub& s)
{
    base::operator=(s); // base part copied
    b = s.b;           // derived part copied
}
```

→ Both base and derived data are copied.

B. User-defined without base call

```
void sub::operator=(sub& s)
{
    b = s.b; // only derived part copied
}
```

→ Base data is **not copied**.

C. User-defined with base call

```
void sub::operator=(sub& s)
{
    b = s.b;
    base::operator=(s); // ensures base copied
}
```

→ Both base and derived parts are copied correctly.

7. Key Observations

- operator= behaves like a constructor, hence **not inherited**.
- operator+, -, *, etc. are inherited like normal functions.
- If not defined in the derived class, compiler's version automatically calls base's operator=.
- If defined manually, programmer must explicitly call base::operator=().
- operator= cannot be inherited or made friend due to its low-level data handling.
- Derived-to-base **object assignment (by value)** leads to **object slicing** (only base part copied).
- When performing addition on derived objects using base's operator+, only base class data** participates — derived members are ignored.
- When the child does not explicitly call base::operator=() inside its assignment operator**, only **derived class data** gets copied — base data remains unchanged.

9. Final Conceptual Summary

- **operator+ and similar operators** → Inherited normally and operate on **base part only** when used through derived objects.
- **operator=** → Not inherited; compiler generates a version for each class.
 - If user defines one, they must call `base::operator=()` explicitly to ensure base data is copied.
 - If not, the compiler's generated version automatically handles both base and derived.

In short:

All overloaded operators behave like normal inherited functions **except** `operator=`, which acts like a constructor and must handle both base and derived parts individually.

When inherited `operator+` is used by derived objects, **only the base data participates** in the operation.

Copy Constructor Inheritance

1. Default Behavior (Compiler-Generated Copy Constructor)

- If you do **not define** a copy constructor in the derived (child) class, the compiler automatically provides one.
- The compiler-generated **copy constructor of the derived class** automatically **calls the copy constructor of the base class**.
- This ensures that both **base and derived data members** are copied properly.

Hence:

```
// Implicitly provided copy constructor of sub:  
sub::sub(const sub& ref) : base(ref) { /* copy own members */ }
```

2. User-Defined Copy Constructor in Derived Class (Without Base Call)

- When you define a copy constructor in the derived class but **do not explicitly call** the base class copy constructor, the **base class's default constructor** is called instead.
- This means the **base portion is not copied correctly** (only initialized by default).
- The derived portion (its own members) gets copied, but the base data remains uninitialized or default-initialized.

Hence: Base data lost → Partial copy.

3. User-Defined Copy Constructor in Derived Class (With Base Call)

- If you explicitly call the **base class's copy constructor** inside the derived class's initialization list, both base and derived members get copied properly.
- This is the **correct and complete** approach when providing a custom copy constructor in the derived class.

Example Pattern:

```
sub::sub(const sub& ref) : base(ref) // explicitly calling base copy  
{  
    b = ref.b;  
}
```

4. Important Rule

- If the **copy constructor is compiler-generated**, it **automatically** calls the **base class copy constructor**.
- If the **copy constructor is user-defined**, you must **explicitly call** the base class copy constructor — otherwise only the derived members are copied, and the base part will invoke its default constructor instead.

5. Core Concept Statement

- The **copy constructor of a derived class must call the copy constructor of its base class** to ensure full member duplication.
- The **compiler-generated version** automatically does this, but if **you define your own**, you **must explicitly call it**.

